

Software Requirements Specification (SRS) per IEEE 830 Standard

1. Introduction

1.1 Purpose

This document describes the functional and non-functional requirements for the Restaurant Ordering System (ROS). The purpose is to define the system's capabilities, performance standards, security measures, user interfaces, and constraints that will meet user needs and expectations.

1.2 Scope

The scope of this document encompasses the ROS functionalities including user registration, login, browsing restaurants, ordering food online, real-time order tracking, multiple payment methods, notifications for delayed orders, menu management by restaurant admins, generating sales reports, and system performance under concurrent users.

1.3 Definitions, Acronyms, and Abbreviations

- ROS: Restaurant Ordering System
- UIs: User Interfaces
- HWIs: Hardware Interfaces
- SWIs: Software Interfaces
- CIs: Communication Interfaces
- API: Application Programming Interface
- UI/UX: User Interface and User Experience
- SSL: Secure Sockets Layer

1.4 References

ANSI/IEEE STD 830-1984

1.5 Overview

The ROS is a web-based application designed to facilitate online ordering of food from restaurants. This system will provide users with an easy-to-use interface to browse menus, place orders, track their orders in real time, and select from various payment methods. Restaurant admins can manage menu items through the admin panel.

2. Overall Description

2.1 Product Perspective

The ROS is a client-server architecture consisting of frontend (UI/UX), backend (API), and database components. Users interact with the frontend, while the backend processes orders and interacts with the database for storing order information.

2.2 Product Functions

- User Registration and Login: Allow users to create accounts and securely log in using their email and password.
- Restaurant Browsing: Enable users to view restaurant menus, descriptions, prices, and contact details.

- Order Placement: Support the addition of items to a cart and finalize orders online.
- Real-time Order Tracking: Allow users to monitor their order status in real time.
- Multiple Payment Methods: Offer credit card payments and digital wallets for making payments securely.
- Notifications: Send email notifications if an order is delayed.
- Menu Management: Provide restaurant admins with the ability to add, update, and remove menu items.
- Sales Reports: Generate daily and monthly sales reports.
- System Performance: Support at least 1000 concurrent users without compromising system performance.

2.3 User Characteristics

- General Public: Users can range from individuals to families, including those with varying technical proficiency levels.
- Restaurant Owners and Managers: Admins responsible for managing menus and sales reports.

2.4 Constraints

- Compatibility: The system must support popular web browsers and mobile devices.
- Security: All payment data must be securely processed and stored using SSL encryption.
- Accessibility: The system should adhere to accessibility standards, such as WCAG 2.1 Level AA.

2.5 Assumptions and Dependencies

- Internet Connectivity: Users require a stable internet connection to use the ROS.
- Payment Gateway Integration: The ROS depends on third-party payment gateways for processing transactions securely.

3. Functional Requirements

- REQ-001: The system should allow users to register and login using their email and password.
- REQ-002: Users should be able to browse restaurants and view menus with prices and descriptions.
- REQ-003: The system should allow users to add items to a cart and place orders online.
- REQ-004: Users should be able to track their orders in real time.
- REQ-005: The system should support multiple payment methods including credit card and digital wallets.
- REQ-006: If an order is delayed, the system should notify the user via email.
- REQ-007: Restaurant admins should be able to add, update, and remove menu items.
- REQ-008: The system should generate daily and monthly sales reports.
- REQ-009: The system should support at least 1000 concurrent users.
- REQ-010: All payment data must be securely processed and stored.

4. Non-Functional Requirements

4.1 Performance Requirements

- Response Time: Respond to search queries within 2 seconds under normal load.
- Scalability: The system should scale horizontally to handle increased traffic and concurrent users.

4.2 Security Requirements

- Data Encryption: Use SSL encryption for secure data transmission between clients and servers.
- Authentication: Implement strong password policies, multi-factor authentication, and account lockout after multiple failed login attempts.

4.3 Usability Requirements

- User Interface (UI) Design: Provide an intuitive and easy-to-navigate UI with clear instructions and error messages.
- Accessibility: Implement accessibility features such as high contrast mode, text size adjustment, and keyboard navigation.

4.4 Reliability Requirements

- Availability: Ensure that the system is available 99% of the time.
- Fault Tolerance: The system should be designed to handle errors gracefully without compromising user experience or data integrity.

4.5 Maintainability Requirements

- Code Documentation: Maintain clear and up-to-date documentation for the entire codebase.
- Testing: Implement unit tests, integration tests, and system tests to ensure the functionality of the ROS.

4.6 Portability Requirements

- Cross-Platform Compatibility: Ensure the ROS supports popular web browsers and mobile devices.
- Platform Migration: The ROS should be designed to migrate easily between different hosting platforms if needed.

5. External Interface Requirements

5.1 User Interfaces (UIs)

- Web Browsers: Support the latest versions of popular web browsers such as Chrome, Firefox, Safari, and Edge.
- Mobile Devices: Compatibility with Android and iOS devices.

5.2 Hardware Interfaces (HWIs)

N/A (No specific HWIs required for this system.)

5.3 Software Interfaces (SWIs)

- API Gateway: Implement an API gateway to manage incoming requests from clients and distribute them to the appropriate services.
- Payment Gateway: Integrate third-party payment gateways for secure payment processing.

5.4 Communication Interfaces (CIs)

- RESTful APIs: Utilize RESTful APIs for communication between the frontend, backend, and database components.

6. System Constraints

- Storage Space: Provide adequate storage space for storing user data, menu items, orders, and sales reports.
- Bandwidth: Ensure sufficient bandwidth to handle high traffic during peak hours.

7. Assumptions and Dependencies

- Internet Connectivity: Users require a stable internet connection to use the ROS.
- Payment Gateway Integration: The ROS depends on third-party payment gateways for processing transactions securely.

This SRS document is created following IEEE 830 standards and provides the functional and non-functional requirements for the Restaurant Ordering System (ROS).